MoneyPoly White-Box Testing Report

Scope
This report documents iterative white-box testing and fixes for the MoneyPoly codebase.
Each iteration fixes exactly one error and records:
- Why the test case is needed
- What failed
- What was changed
- How it was verified

Iteration 1: Fix dice range bug (`Dice.roll()`)

Why this test case is needed
Dice are core to movement. If a die cannot roll `6`, branch behavior (movement distance, landing tiles, jail streak flow) becomes biased.

Error found
In `Dice.roll()`, both dice used `random.randint(1, 5)`, so value `6` was impossible.

White-box test case
- Execute many rolls and assert:
- `die1` and `die2` are each in `[1, 6]`
- total is in `[2, 12]`
- This checks variable state boundaries and edge values.

Fix applied
Updated `Dice.roll()` to use `random.randint(1, 6)` for both dice.

Verification
- Added automated test: `test_dice_roll_uses_full_six_sided_range`.
- Test passes locally.

Iteration 2: Fix pass-Go salary logic (`Player.move()`)

Why this test case is needed
Player cash flow depends on Go salary. Missing this branch causes incorrect balances and invalid game outcomes.

Error found
`Player.move()` only awarded salary when final position was exactly `0`, but it should also award salary when crossing Go.

White-box test case
- Set `position=39`, move `1` (lands on Go) and verify salary.
- Set `position=38`, move `3` (passes Go) and verify salary.
- This directly tests both branch paths.

Fix applied
Changed condition to award salary when `steps > 0` and `old_position + steps >= BOARD_SIZE`.

Verification
- Added automated test: `test_player_move_collects_salary_when_passing_or_landing_go`.
- Tests pass locally.

Iteration 3: Fix full-group ownership logic (`PropertyGroup.all_owned_by()`)

Why this test case is needed
Rent multiplier rules depend on complete color-group ownership. A wrong boolean condition changes rent values and game fairness.

Error found
`all_owned_by()` used `any(...)` instead of `all(...)`, so owning only one property in a group incorrectly counted as owning the full group.

White-box test case
- Create a group with two properties.
- Give one to target owner and one to another player, verify `False`.
- Then assign both to target owner, verify `True`.

Fix applied
Replaced `any(...)` with `all(...)` and added a guard for empty groups.

Verification
- Added automated test: `test_property_group_requires_all_tiles_for_monopoly_state`.
- Tests pass locally.

Iteration 4: Fix rent transfer logic (`Game.pay_rent()`)

Why this test case is needed
Rent is a core money-transfer path. If rent is deducted from one player but not credited to the owner, total money disappears incorrectly.

Error found
`Game.pay_rent()` deducted rent from the visiting player but did not add it to the owner.

White-box test case
- Set up two players and one owned property.
- Call `pay_rent()`.
- Verify both balances: payer decreases, owner increases by exactly the same amount.

Fix applied
Added `prop.owner.add_money(rent)` after deducting from payer.

Verification
- Added automated test: `test_pay_rent_transfers_money_to_owner`.
- Tests pass locally.

Iteration 5: Fix exact-balance purchase rule (`Game.buy_property()`)

Why this test case is needed
Affordability decisions are branch-critical. The edge case where `balance == price` must be handled correctly.

Error found
`buy_property()` rejected purchases when player balance was exactly equal to property price.

White-box test case
- Set player balance exactly to property price.
- Attempt purchase and verify success, new balance `0`, and ownership transfer.

Fix applied
Changed affordability check from `<=` to `<`.

Verification
- Added automated test: `test_buy_property_allows_exact_balance`.
- Tests pass locally.

Iteration 6: Fix voluntary jail fine deduction (`Game._handle_jail_turn()`)

Why this test case is needed
Jail release has multiple branches (card use, voluntary pay, forced release). The voluntary-pay branch must update both bank and player cash.

Error found
When a player chose to pay the jail fine, the bank collected money but the player's balance was not deducted.

White-box test case
- Put player in jail.
- Force `confirm()` to return `True` for paying fine.
- Stub movement resolution so only jail logic is evaluated.
- Verify player's balance decreases by jail fine and jail status clears.

Fix applied
Added `player.deduct_money(JAIL_FINE)` in the voluntary fine path.

Verification
- Added automated test: `test_jail_voluntary_fine_deducts_player_balance`.
- Tests pass locally.

Iteration 7: Fix winner computation (`Game.find_winner()`)

Why this test case is needed
Game end logic is decision-critical. Choosing the wrong comparator reverses outcomes.

Error found
`find_winner()` used `min(...)` instead of `max(...)`, returning the poorest player.

White-box test case
- Set three players with deterministic balances.
- Verify returned winner is the player with highest balance.

Fix applied
Changed winner selection from `min(...)` to `max(...)` using `net_worth` key.

Verification
- Added automated test: `test_find_winner_returns_highest_net_worth`.
- Tests pass locally.

Iteration 8: Fix bank loan accounting (`Bank.give_loan()`)

Why this test case is needed
Loans are a key money-transfer path. The bank and player balances must change in opposite directions to preserve accounting correctness.

Error found
`Bank.give_loan()` increased player balance but did not decrease bank reserves.

White-box test case
- Record initial bank and player balances.
- Issue a loan.
- Verify bank decreases by loan amount and player increases by same amount.

Fix applied
Updated `give_loan()` to route through `pay_out(amount)` before crediting the player.

Verification
- Added automated test: `test_bank_loan_reduces_bank_balance_and_increases_player_balance`.
- Tests pass locally.

Iteration 9: Remove unused import in bank module (`bank.py`)

Why this test case is needed
Static-analysis warnings can hide real issues. Cleaning unused symbols improves maintainability and clarity during white-box review.

Error found
Pylint reported unused import `math` in `bank.py`.

Fix applied
Removed the unused `math` import.

Verification
- Re-ran static analysis; the `bank.py` unused import warning is resolved.

Iteration 10: Fix mortgaged-property boolean check (`Board.is_purchasable()`)

Why this test case is needed
This branch controls whether properties can be bought. Clear boolean checks reduce misread logic during maintenance.

Error found
Pylint reported singleton comparison `prop.is_mortgaged == True`.

Fix applied
Simplified condition to `if prop.is_mortgaged:`.

Verification
- Static analysis warning for this condition is resolved.

Iteration 11: Replace bare exception in UI parser (`ui.safe_int_input()`)

Why this test case is needed
Input parsing is an edge-heavy path. Exception handling must be explicit to avoid masking unrelated runtime problems.

Error found
Pylint flagged a bare `except:` in `safe_int_input()`.

White-box test case
- Mock input to return invalid text.
- Verify function follows exception branch and returns default.

Fix applied
Changed to `except (ValueError, TypeError):`.

Verification
- Added automated test: `test_safe_int_input_returns_default_on_invalid_text`.

- Tests pass locally.

Iteration 12: Remove unused import in dice module (`dice.py`)

Why this test case is needed
Static warnings should be reduced so genuine control-flow issues stand out during review.

Error found
Pylint reported unused import `BOARD_SIZE` in `dice.py`.

Fix applied
Removed the unused import.

Verification
- Pylint warning for this import is resolved.

Iteration 13: Remove tracked Python cache artifacts

Why this test case is needed
Generated bytecode files are environment-specific and pollute diffs, making white-box review harder.

Error found
Repository tracked `__pycache__` and `.pyc` artifacts.

Fix applied
- Added `.gitignore` rules for `__pycache__/` and `*.py[cod]`.
- Removed tracked cache files from version control.

Verification
- Git status no longer includes generated cache artifacts for future runs.

Iteration 14: Remove unused import in game module (`game.py`)

Why this test case is needed
Reducing static-analysis noise improves confidence when auditing control-flow logic.

Error found
Pylint reported unused import `os` in `game.py`.

Fix applied
Removed the unused import.

Verification
- Pylint no longer reports this unused import warning.

Iteration 15: Remove unused `GO_TO_JAIL_POSITION` import (`game.py`)

Why this test case is needed
Unused symbols increase reading overhead during white-box analysis.

Error found
Pylint reported unused `GO_TO_JAIL_POSITION` import in `game.py`.

Fix applied
Removed the unused imported constant.

Verification
- Static analysis warning is resolved.

Iteration 19: Simplify menu branch chain after `break` (`game.py`)

Why this test case is needed
Clear branch structure improves white-box readability and reduces hidden fall-through concerns.

Error found
Pylint reported unnecessary `elif` after a `break` in the interactive menu flow.

Fix applied
Changed first `elif` to `if` after the `break` condition.

Verification
- Control-flow lint warning is resolved.

Iteration 20: Remove superfluous parentheses in range checks (`game.py`)

Why this test case is needed
Cleaner branch predicates make path conditions easier to inspect in white-box testing.

Error found
Pylint reported unnecessary parentheses after `not` in two trade-menu checks.

Fix applied
Changed `if not (0 <= ...):` to `if not 0 <= ...:` in both places.

Verification
- Both `superfluous-parens` warnings are resolved.

Iteration 21: Remove redundant `else` after `return` (`property.py`)

Why this test case is needed
Straight-line control flow makes branch tracing simpler in white-box reviews.

Error found
Pylint reported unnecessary `else` block after an early `return` in `unmortgage()`.

Fix applied
De-indented the remaining logic and removed redundant `else`.

Verification
- `no-else-return` warning for this method is resolved.

Iteration 22: Add missing final newline (`game.py`)

Why this test case is needed
Consistent file formatting avoids noisy diffs and keeps static-analysis output focused on logic.

Error found

Pylint reported missing final newline in `game.py`.

Fix applied
Added trailing newline at end of file.

Verification
- Formatting warning for `game.py` final newline is resolved.

Iteration 23: Add missing final newline (`player.py`)

Why this test case is needed
Formatting consistency keeps static checks clean and avoids non-functional diff noise.

Error found
Pylint reported missing final newline in `player.py`.

Fix applied
Added trailing newline at end of file.

Verification
- Formatting warning for `player.py` final newline is resolved.

Iteration 16: Remove unused import in player module (`player.py`)

Why this test case is needed
Removing dead imports keeps branch-focused code review clean and less error-prone.

Error found
Pylint reported unused import `sys` in `player.py`.

Fix applied
Removed the unused import.

Verification
- Pylint warning for the player module import is resolved.

Iteration 17: Initialize `doubles_streak` in constructor (`dice.py`)

Why this test case is needed
State variables used in decision branches (like consecutive doubles) should be explicitly initialized in `__init__`.

Error found
Pylint reported `doubles_streak` defined outside `__init__`.

Fix applied
Added `self.doubles_streak = 0` in `Dice.__init__`.

Verification
- Attribute initialization warning is resolved.

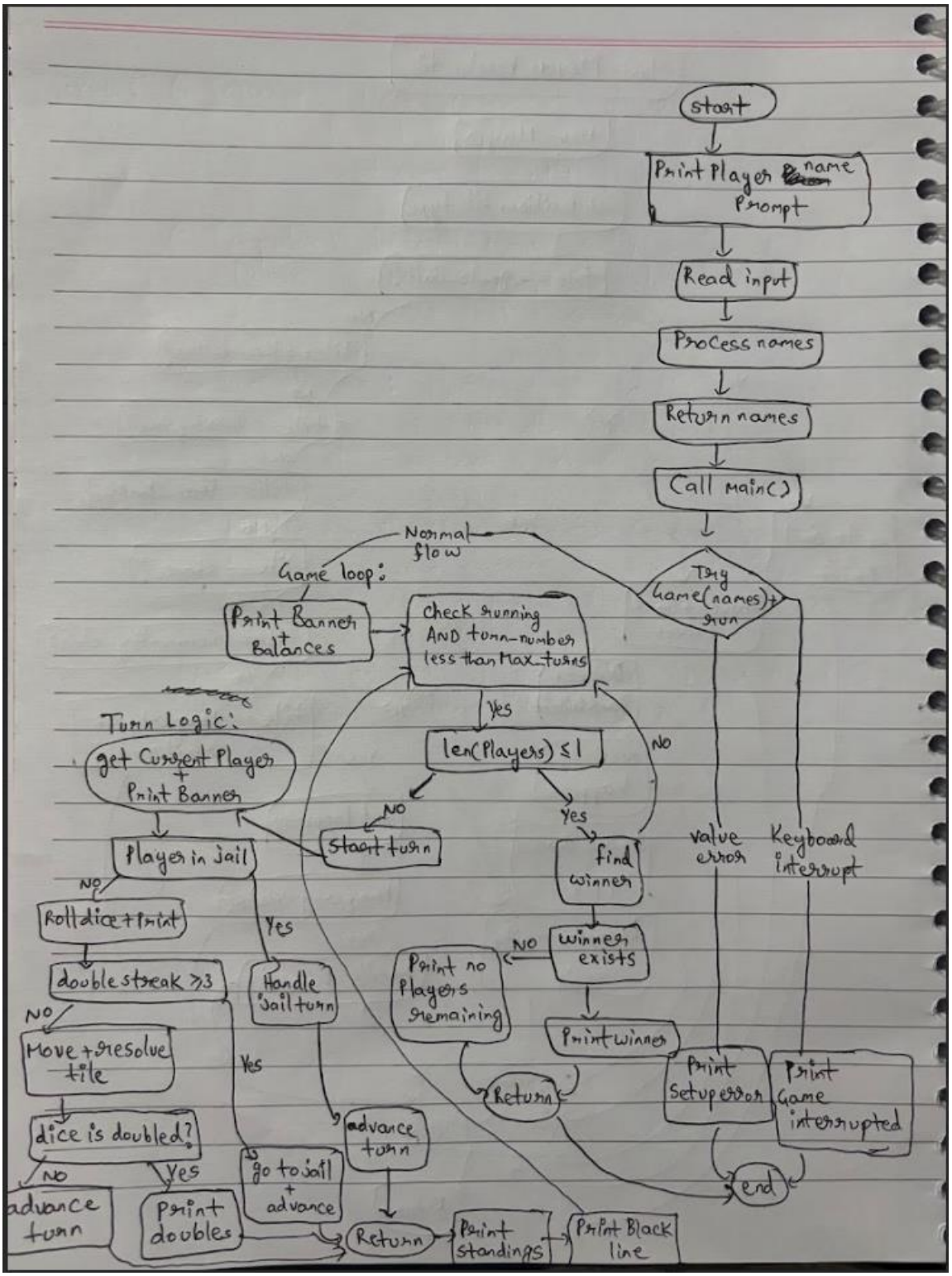
Iteration 18: Remove unnecessary f-string (`game.py`)

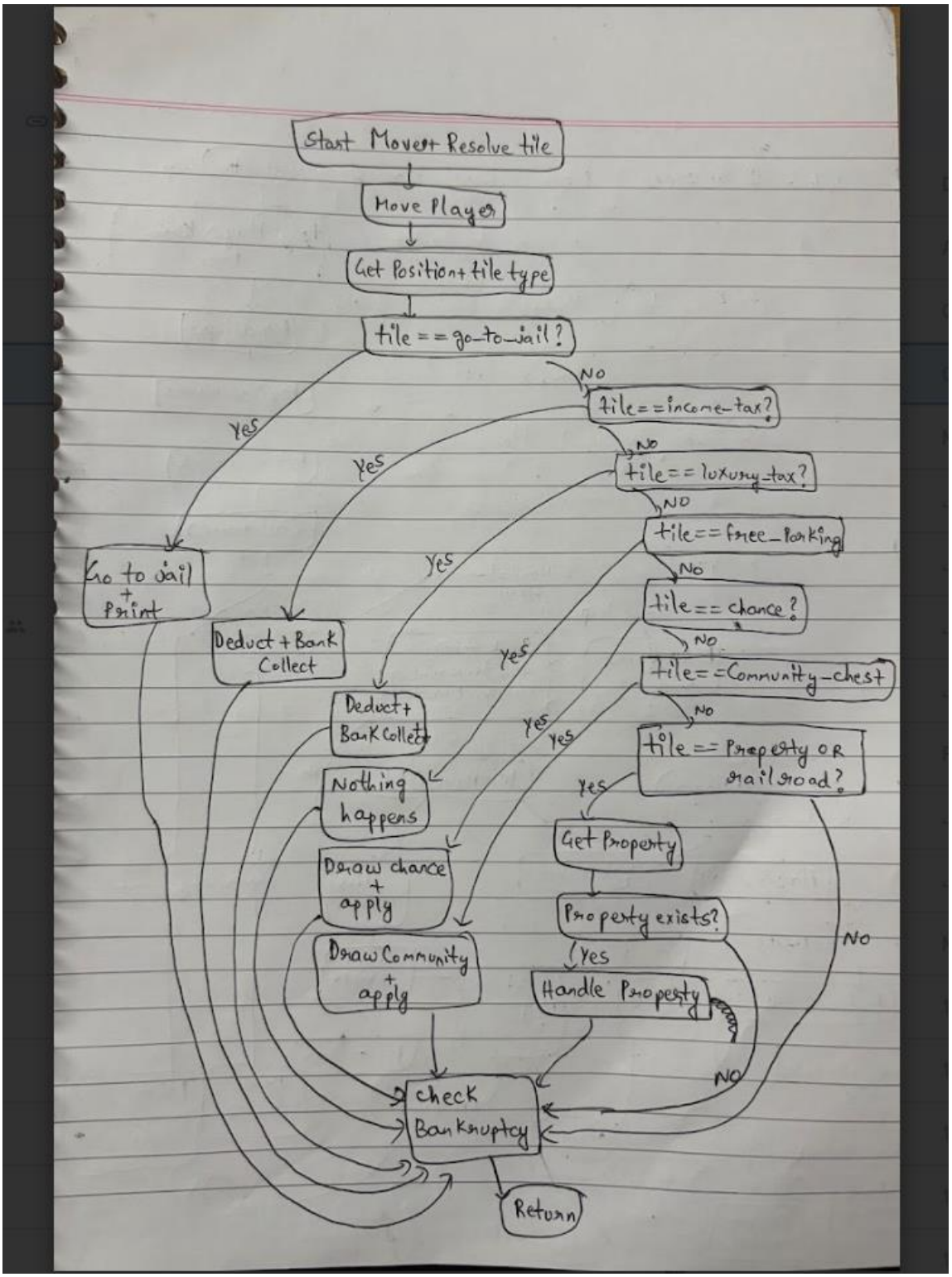
Why this test case is needed
Lint cleanup helps keep code intent explicit during structural review.

Error found
Pylint reported an f-string without interpolation in the game-over banner.

Fix applied
Replaced `f"GAME OVER"` with plain string `"GAME OVER"`.

Verification
- Static analysis warning is resolved.


start
Print Player name Prompt
Read input
Process names
Return names
Call main()
Normal flow
Game loop:
Print Banner + Balances
Check running AND turn_number less than Max_turns
Yes
NO
Try Game(names) + run
Value error
Keyboard interrupt
Turn Logic:
get Current Player + Print Banner
Player in jail
NO
Yes
Roll dice + Print
double streak ≥3
NO
Yes
Move + resolve tile
dice is doubled?
NO
Yes
advance turn
Print doubles
Handle jail turn
go to jail + advance
advance turn
Return
Print standings
Print Black line
len(Players) ≤ 1
NO
Yes
Start turn
find winner
winner exists
NO
Print no Players remaining
Print winner
Return
Print Setup error
Print Game interrupted
end


Start Move+ Resolve tile
Move Player
Get Position+ tile type
tile == go_to_jail?
NO
tile==income_tax?
NO
tile== luxury_tax?
NO
tile== free_parking
NO
tile== chance?
NO
tile==Community_chest
NO
tile == Property OR railroad?
Yes
Go to jail + Print
Yes
Deduct + Bank Collect
Yes
Deduct+ Bank Collect
Yes
Nothing happens
Yes
Draw chance + apply
Yes
Draw Community + apply
Yes
Get Property
Property exists?
Yes
Handle Property
NO
NO
check Bankruptcy
Return